1 SQlite数据库

1、数据库：

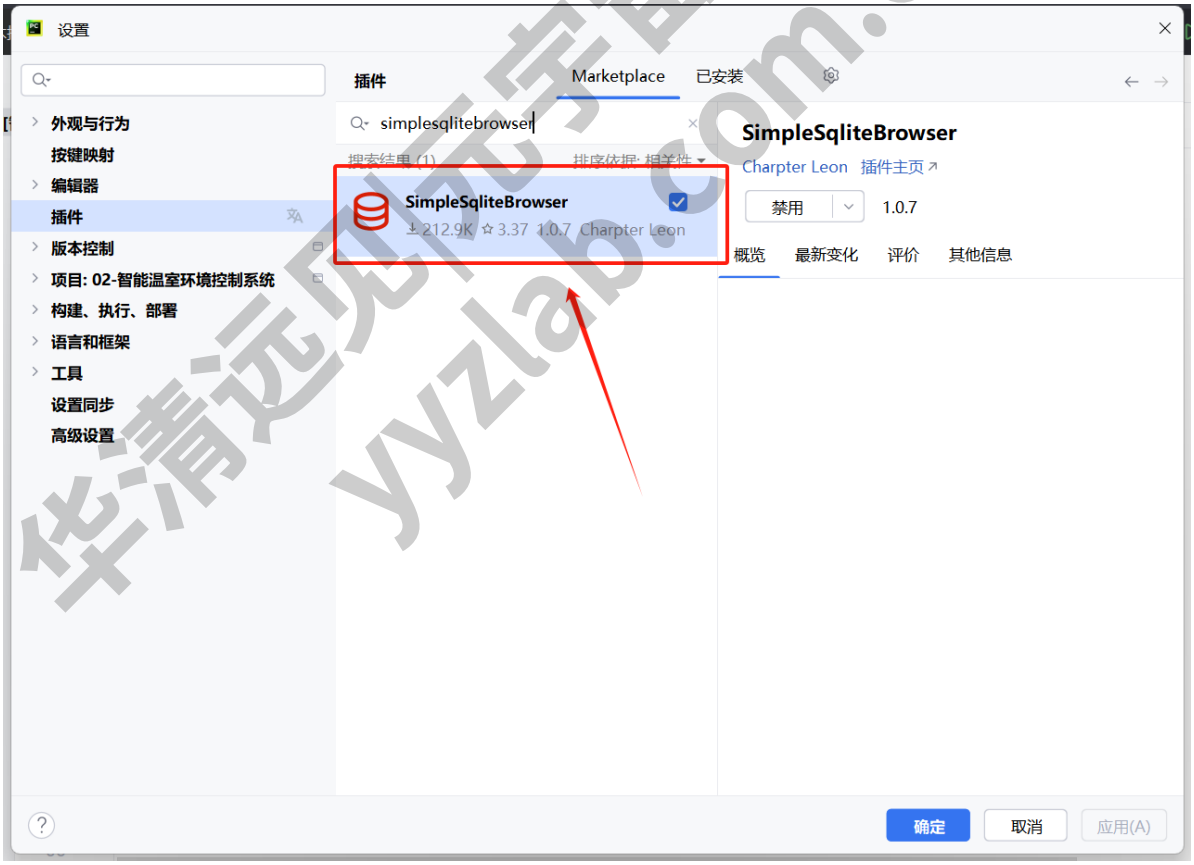
数据库是一个**组织和存储**数据的集合，它提供了一种结构化和有组织的方式来管理和访问数据。数据库可以存储大量的数据，并提供快速、安全、可靠的数据操作和查询。常见的数据库软件有MySQL、Oracle、SQL Server、PostgreSQL等。这些软件具有管理、查询、更新、删除数据的功能，可以通过编写**SQL语句**来操作数据库中的数据。

2、SQLite数据库

- SQLite 是一种轻量级的关系型数据库管理系统，它是以磁盘文件形式存储数据的嵌入式数据库引擎。
- 与传统的数据库管理系统不同，SQLite 不需要单独的服务器进程，它直接访问存储在文件中的数据，以单一的文件形式存在。
- SQLite 的设计目标是简单、轻巧、高效，因此它被广泛应用于嵌入式系统和小型应用程序中。

PyCharm插件安装

```
文件--设置--插件--Marketplace
安装SimpleSqliteBrowser
```



PyQT5中有自带的数据库模块-QtSql:

使用的时候需要引入如下模块：

```
from PyQt5 import QtSql
from PyQt5.QtSql import QSqlQuery
```

华清远见元宇宙实验中心 yyzlab.com.cn

其中：

- QtSql类即QT中的QSqlDatabase类，用于处理与数据库的连接
- QSqlQuery类提供了执行和操作SQL语句的方法

使用QSqlDatabase连接数据库，如果没有则新建一个数据库，可指定路径

```
from PyQt5.QtCore import Qt
from PyQt5.QtSql import QSqlDatabase, QSqlQuery

# 创建 SQLite 数据库连接
def createConnection():
    db = QSqlDatabase.addDatabase("QSQLITE")
    db.setDatabaseName("SensorData.db")  # 修改为新的数据库文件名
    if not db.open():
        print("无法建立数据库连接！")
        return False
    return True
```

建表、增、删、改、查：

```
# 创建一个Sql查询的对象
query = QSqlQuery()

# 在数据库user.db中，创建一个新表，表头为ID，username与password
# 没有自动排序  ID是一个整数类型的列，并且被设置为主键。用于唯一标识表中的每行数据。username text和password text：这两个列被定义为文本类型。
query.exec_("create table if not exists test_info(ID int primary key, username text, password text) ")
# 有自动排序   ID是一个整数类型的列，并且被设置为主键。与第一个命令不同，这里AUTOINCREMENT被添加到主键定义中。这意味着ID列将自动递增
query.exec_("create table if not exists user_info(ID integer primary key AUTOINCREMENT, username text, password text) ")
```

```
# 查询数据库
def queryDatabase():
    query = QSqlQuery()
    query.exec_("SELECT * FROM sensor")  # 修改为新的表名
    while query.next():
        id = query.value(0)  # 获取列值

# 更新数据库
def updateDatabase():
    query = QSqlQuery()
    query.exec_("UPDATE sensor SET column1 = 'new value' WHERE id = 1")
```

```python
# 插入数据
def insertData():
    query = QSqlQuery()
    query.prepare("INSERT INTO sensor (column1, column2) VALUES (?, ?)")
    query.bindValue(0, "value1")  # 绑定参数值
    query.bindValue(1, "value2")
    query.exec_()  # 执行插入语句

# 删除数据
def deleteData():
    query = QSqlQuery()
    query.exec_("DELETE FROM sensor WHERE id = 1")  # 修改为新的表名

# 关闭数据库连接
def closeConnection():
    db = QSqlDatabase.database()
    db.close()
```

2 总结

2.1 QSqlQuery

在 PyQt5 中，`QSqlQuery` 是一个用于执行 SQL 查询的类。它属于 `PyQt5.QtSql` 模块，允许你与数据库进行交互，执行 SQL 语句（如 `SELECT`、`INSERT`、`UPDATE`、`DELETE` 等），并处理查询结果。

`QSqlQuery` 的主要作用是执行 SQL 查询并与数据库进行交互。它可以用于以下场景：

- **执行 SQL 查询**：如 `SELECT`、`INSERT`、`UPDATE`、`DELETE` 等。
- **获取查询结果**：可以遍历查询结果集，获取每一行的数据。
- **绑定参数**：支持使用占位符来防止 SQL 注入攻击。
- **事务管理**：可以结合事务机制来确保数据的一致性。

2.1.1 QSqlQuery 的导入方式

```python
from PyQt5.QtSql import QSqlQuery
```

2.1.2 常用方法

2.1.2.1 执行 SQL 查询

- `exec_(str)`：执行一条 SQL 查询语句。

```python
query = QSqlQuery()
query.exec_("SELECT * FROM users")
```

华清远见北京研发中心 yyzlab.com.cn

2.1.2.2 绑定参数

- `prepare(str)`：准备一个带有占位符的 SQL 语句。
- `bindValue(placeholder, value)`：为占位符绑定具体的值。
- `exec_()`：执行带有绑定参数的 SQL 语句。

```
query.prepare("INSERT INTO users (name, age) VALUES (:name, :age)")
query.bindValue(":name", "Alice")
query.bindValue(":age", 25)
query.exec_()
```

2.1.2.3 遍历查询结果

- `next()`：移动到结果集的下一行。
- `value(int)`：获取当前行中指定列的值。
- `record()`：获取当前行的记录对象，可以通过字段名或索引访问数据。

```
query.exec_("SELECT name, age FROM users")
while query.next():
    name = query.value(0)  # 获取第一列的值
    age = query.value(1)   # 获取第二列的值
    print(f"Name: {name}, Age: {age}")
```

2.1.2.4 事务管理

- `transaction()`：开始一个事务。
- `commit()`：提交事务。
- `rollback()`：回滚事务。

```
db.transaction()
query.exec_("INSERT INTO users (name, age) VALUES ('Bob', 30)")
if some_condition:
    db.commit()  # 提交事务
else:
    db.rollback()  # 回滚事务
```

2.1.2.5 检查查询是否成功

- `isActive()`：检查查询是否成功执行。
- `lastError()`：获取查询执行过程中发生的错误信息。

```
if not query.exec_("SELECT * FROM users"):
    print(query.lastError().text())
```

2.1.3 典型应用场景

- **执行 SQL 查询**：如从数据库中读取数据、插入新记录、更新现有记录、删除记录等。
- **绑定参数**：通过占位符绑定参数，防止 SQL 注入攻击。
- **遍历查询结果**：从查询结果集中获取每一行的数据，并进行处理。
- **事务管理**：确保多个数据库操作要么全部成功，要么全部失败。

华清远见|元宇宙实验中心
yyzlab.com.cn

2.2 QSqlDatabase

在 PyQt5 中，`QSqlDatabase` 是一个用于管理数据库连接的类。它属于 `PyQt5.QtSql` 模块，允许你与不同的数据库系统（如 SQLite、MySQL、PostgreSQL 等）进行交互。通过 `QSqlDatabase`，你可以创建、配置和管理数据库连接，并执行 SQL 查询。

`QSqlDatabase` 的主要作用是管理和维护数据库连接。它负责以下任务：

- **创建数据库连接**：可以连接到不同类型的数据库（如 SQLite、MySQL、PostgreSQL 等）。
- **配置数据库连接**：设置数据库的名称、主机、端口、用户名、密码等信息。
- **打开和关闭数据库连接**：控制数据库连接的生命周期。
- **事务管理**：支持事务机制，确保数据的一致性。
- **多数据库支持**：可以同时管理多个数据库连接。

2.2.1 QSqlDatabase` 的导入方式

```
from PyQt5.QtSql import QSqlDatabase
```

2.2.2 常用方法

2.2.2.1 添加数据库连接

- `addDatabase(str driver, str connectionName)`：创建一个新的数据库连接。
  - `driver`：指定数据库驱动程序，例如 `"QSQLITE"`（SQLite）、`"QMYSQL"`（MySQL）、`"QPSQL"`（PostgreSQL）等。
  - `connectionName`（可选）：为连接指定一个唯一的名称。如果未指定，默认使用默认连接。

```
db = QSqlDatabase.addDatabase("QSQLITE")  # 创建一个 SQLite 数据库连接
```

2.2.2.2 设置数据库连接参数

- `setDatabaseName(str)`：设置数据库文件的名称（对于 SQLite）或数据库名（对于其他数据库）。
- `setHostName(str)`：设置数据库服务器的主机名（适用于 MySQL、PostgreSQL 等）。
- `setPort(int)`：设置数据库服务器的端口号。
- `setUserName(str)`：设置数据库的用户名。
- `setPassword(str)`：设置数据库的密码。

```
db.setDatabaseName("example.db")  # 对于 SQLite，设置数据库文件名
db.setHostName("localhost")       # 设置主机名（适用于 MySQL、PostgreSQL 等）
db.setPort(3306)                  # 设置端口号（适用于 MySQL、PostgreSQL 等）
db.setUserName("root")            # 设置用户名
db.setPassword("password")        # 设置密码
```

2.2.2.3 打开和关闭数据库连接

- `open()`：打开数据库连接。
- `close()`：关闭数据库连接。
- `isOpen()`：检查数据库连接是否已打开。

华清远见元宇宙实验中心 yyzlab.com.cn

```
if not db.open():
    print("无法打开数据库")
    sys.exit(1)

# 关闭数据库连接
db.close()
```

2.2.2.4 事务管理

- `transaction()`：开始一个事务。
- `commit()`：提交事务。
- `rollback()`：回滚事务。

```
db.transaction()
query.exec_("INSERT INTO users (name, age) VALUES ('Bob', 30)")
if some_condition:
    db.commit()  # 提交事务
else:
    db.rollback()  # 回滚事务
```

2.2.2.5 获取数据库驱动信息

- `drivers()`：返回当前可用的数据库驱动列表。
- `isDriverAvailable(str)`：检查指定的数据库驱动是否可用。

```
print(QSqlDatabase.drivers())  # 打印所有可用的数据库驱动
if QSqlDatabase.isDriverAvailable("QSQLITE"):
    print("SQLite 驱动可用")
```

2.2.2.6 多数据库连接

- `database(str connectionName)`：根据连接名称获取特定的数据库连接。
- `removeDatabase(str connectionName)`：移除指定的数据库连接。

```
db1 = QSqlDatabase.addDatabase("QSQLITE", "connection1")
db2 = QSqlDatabase.addDatabase("QMYSQL", "connection2")

# 获取特定的数据库连接
db1 = QSqlDatabase.database("connection1")
db2 = QSqlDatabase.database("connection2")

# 移除数据库连接
QSqlDatabase.removeDatabase("connection1")
```

2.2.3 典型应用场景

- **连接数据库**：连接到 SQLite、MySQL、PostgreSQL 等数据库。
- **执行 SQL 查询**：通过 `QSqlQuery` 执行 SQL 查询（如 `SELECT`、`INSERT`、`UPDATE`、`DELETE` 等）。
- **事务管理**：确保多个数据库操作要么全部成功，要么全部失败。
- **多数据库支持**：同时管理多个数据库连接。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见|元宇宙实验中心
yyzlab.com.cn